

SKIM

Document Extraction Application

Technical Documentation & Product Roadmap

A complete technical and functional guide to the architecture, workflows, extraction engine, Qt interface, project structure, reliability features, and release plan of Skim.

Version 1.0 • September 2026

Windows • C++17 • Qt 6.11.2 • MinGW • CMake

Table of Contents

1	Project Overview
2	Goals and Design Philosophy
3	Functional Scope
4	System Architecture
5	Application Workflow
6	Input Modes
7	Supported Document Formats
8	Text Extraction Details
9	Word Document Processing
10	PDF Processing
11	Output Generation
12	Background Processing and Cancellation
13	Progress, Logging, and Statistics
14	Error Handling
15	Graphical User Interface
16	Drag-and-Drop and File Selection
17	Menus and Keyboard Shortcuts
18	Branding and Product Identity
19	Build System and Development Environment
20	Project Structure
21	Testing and Validation
22	Current Roadmap
23	Release and Packaging Plan
24	Future Enhancements
25	Known Deliberate Non-Features
26	Developer Notes and Maintenance Guidance

1. Project Overview

Skim is a Windows desktop document-extraction application developed in C++ with Qt. Its purpose is to replace a previous VBA-based document scanning workflow with a dedicated application that can search documents for a user-supplied term and produce a consolidated text output containing the matching lines and relevant document information.

The application is designed around a simple principle: the user chooses what to scan, enters what to search for, starts extraction, and receives both a human-readable log and a structured output summary. The implementation separates the user interface from the extraction worker so document processing does not freeze the GUI.

The project began as **DocumentExtractor** and was subsequently productized under the name **Skim**. Internal C++ class names may retain the original DocumentExtractor terminology where doing so avoids unnecessary refactoring.

2. Goals and Design Philosophy

The project roadmap follows a progression from a reliable replacement for the original macro to a polished standalone desktop product.

- **Correctness first:** support the document types needed by the original workflow and preserve predictable matching behavior.
- **Responsiveness:** perform extraction away from the GUI thread and provide progress and cancellation.
- **Usability:** support both folder-oriented and individual-file workflows, including drag-and-drop.
- **Transparency:** report what was scanned, what matched, what was empty, and what failed.
- **Product quality:** provide a consistent Qt stylesheet, application icon, menus, shortcuts, dialogs, and branding.
- **Maintainability:** keep the worker, UI, build configuration, resources, and application entry point clearly separated.

3. Functional Scope

Capability	Status	Description
TXT extraction	Complete	Search matching lines in text files.
Word extraction	Complete	Process .doc and .docx using Microsoft Word automation through ActiveQt.
PDF extraction	Complete	Extract text page-by-page with Qt PDF.
Case-insensitive search	Complete	Matches are performed without case sensitivity.
Folder scanning	Complete	Scan a directory with optional recursive subfolder traversal.
Multi-file scanning	Complete	Select or drag multiple supported documents, including files from different locations.
Cancellation	Complete	Cooperative cancellation between processing steps.

Capability	Status	Description
Progress/logging	Complete	Live progress and extraction messages.
Statistics	Complete	Files found/scanned, matches, empty files, and failures.
Failure reporting	Complete	Failed file paths are collected and shown in the completion summary.
Polished GUI	Complete	Card layout, QSS styling, menus, shortcuts, drag-and-drop, icon.

4. System Architecture

At a high level, Skim is divided into a Qt GUI layer, an extraction worker, format-specific processing routines, output/reporting logic, and application resources.

```
MainWindow | | starts/configures v DocumentExtractorWorker | +-- TXT processor | +-- Word processor
(ActiveQt / Microsoft Word) | +-- PDF processor (QtPdf / QPdfDocument) | +-- output writer | +--
progress / log signals | +-- cooperative cancellation | v Output file + completion summary
```

The main window owns the user-facing state and controls. The worker performs the potentially expensive extraction work. Signals and slots are used to communicate progress and completion without requiring the GUI to perform document processing itself.

5. Application Workflow

- 1 The user launches Skim.
- 2 The user chooses Folder mode or Files mode.
- 3 The user selects a folder or one/more supported files.
- 4 The user enters the search term.
- 5 The user chooses/configures the output destination.
- 6 The user starts extraction.
- 7 The GUI starts the worker and displays progress/log messages.
- 8 The worker enumerates the selected inputs and dispatches each file to its format-specific processor.
- 9 Matching lines are written to the output stream and counted.
- 10 Empty files and failed files are tracked separately.
- 11 The user may request cancellation; the worker checks the cancellation flag between processing operations.
- 12 At completion, Skim presents a summary containing scan statistics and failed-file information.

6. Input Modes

Folder mode is intended for the original directory-oriented workflow. The user supplies one folder and controls whether subfolders are included. Recursive traversal allows a directory tree to be processed without manually

selecting each file.

Files mode is intended for targeted extraction. The user may select one or many documents. Multiple drag-and-drop operations are additive, duplicate paths are ignored, and files may originate from different folders or drives. Switching back to Folder mode clears the selected-file list because the two modes represent different scan sources.

7. Supported Document Formats

Extension	Processor	Notes
.txt	Native text processing	Lines are searched directly.
.doc	Microsoft Word via ActiveQt	Requires the Word automation environment used by the application.
.docx	Microsoft Word via ActiveQt	Opened through Word's Documents.Open automation API.
.pdf	QtPdf	Text PDFs are processed page-by-page.

The current PDF implementation is text extraction rather than OCR. Image-only PDFs therefore do not become searchable simply because they are PDFs; OCR would be a separate future feature.

8. Text Extraction Details

For line-oriented formats, Skim searches individual lines and records lines containing the search term. Matching uses Qt's case-insensitive string search behavior. Empty or whitespace-only lines are skipped where appropriate.

```
if (line.contains(searchText, Qt::CaseInsensitive)) { writeMatch(output, filePath, line);  
++matchesFound; }
```

The actual implementation also performs trimming/empty-line checks and integrates cancellation checks. This approach intentionally favors understandable, predictable matching over a complex search language.

9. Word Document Processing

Word documents are handled through Qt ActiveQt/QAxContainer and Microsoft Word automation. The worker creates a Word automation object, configures Word to remain hidden, opens documents through the Word Documents collection, extracts their textual content, and closes the document. Word is started once for an extraction run and shut down when processing ends.

A known working opening signature is:

```
QAxObject *documents = word->querySubObject("Documents"); QAxObject *document =  
documents->querySubObject( "Open(const QString&, bool)", nativePath, true );
```

Keeping Word hidden and disabling unnecessary interactive alerts is important for unattended extraction. The worker also closes the Word document/application during cleanup, including the cancellation path.

10. PDF Processing

PDFs are loaded with Qt's QPdfDocument. The worker iterates through every page and obtains the page's complete text using QPdfSelection. Text is split into lines, empty lines are ignored, and matching lines are written with a page number.

```
QPdfDocument document; QPdfDocument::Error error = document.load(filePath); if (error !=
QPdfDocument::Error::None) return false; for (int page = 0; page < document.pageCount(); ++page) {
QPdfSelection selection = document.getAllText(page); QString pageText = selection.text(); // split
into lines and search case-insensitively }
```

The page number stored in the output is one-based, so a match on internal page index 0 is reported as page 1.

11. Output Generation

Skim writes matches to a text output file and finishes with a structured extraction summary. The summary distinguishes between files discovered, files successfully scanned, matches, empty files, and failures.

```
===== EXTRACTION SUMMARY =====
Files found: ... Files scanned: ... Matches found: ... Empty files: ... Failed files: ... FAILED
FILES -----
```

When a run is cancelled, the completion header identifies the operation as **EXTRACTION CANCELLED** rather than presenting the operation as a normal successful completion.

12. Background Processing and Cancellation

Document extraction can involve filesystem access, PDF parsing, and Word automation. Performing these operations directly in the GUI thread would make the application appear frozen. Skim therefore uses a worker object running independently from the main interface.

Cancellation is cooperative rather than forcibly terminating a thread. A QAtomicInt cancellation flag is checked during processing. The worker can finish the current small operation safely, stop subsequent work, close resources, and report cancellation.

This design is particularly important for Word automation: abruptly killing a thread could leave an automation server or document in an undesirable state. Controlled cancellation allows cleanup code to run.

13. Progress, Logging, and Statistics

The interface exposes live extraction activity through a log area and progress controls. Typical events include scanning a file, completing a file, skipping an empty file, and reporting a failure.

- **Files found:** number of candidate inputs discovered/selected.
- **Files scanned:** number successfully processed.
- **Matches found:** total matching lines recorded.
- **Empty files:** inputs skipped because they contained no useful content.
- **Failed files:** inputs whose processing could not be completed successfully.

14. Error Handling

Skim treats document failures as data to report rather than as a reason to silently terminate the entire run. A processor returns failure for an unreadable/corrupt document or another processing problem. The worker records

that file path in its failed-file collection, allowing the rest of the run to continue where possible.

This behavior is important for batch extraction: one bad document should not hide successful results from dozens or hundreds of other documents.

15. Graphical User Interface

The main window uses a card-oriented layout. Major areas are separated visually so the user can understand the workflow at a glance.

- **Scan Location:** Folder/Files selection and source path(s).
- **Search + Output:** search term and output destination controls.
- **Options:** scan behavior such as subfolder inclusion.
- **Progress:** current operation and progress state.
- **Extraction Log:** live textual activity.
- **Start / Cancel:** primary operation controls.

The application uses a global Qt stylesheet resource for consistent typography, spacing, controls, cards, checkboxes, radio buttons, and buttons. The main window is intentionally fixed at approximately 1150 × 1100 pixels to preserve the designed layout.

16. Drag-and-Drop and File Selection

Drag-and-drop is integrated into the main application workflow. Folder mode accepts a folder source, while Files mode accepts individual supported documents. Files mode permits multiple selections and additive drops.

- Unsupported extensions are rejected rather than treated as valid extraction sources.
- Duplicate file paths are ignored.
- Files from different directories and drives can coexist in one Files-mode selection.
- The file picker supports multi-selection.
- The UI displays a concise selected-document count rather than attempting to display every path in the primary field.

17. Menus and Keyboard Shortcuts

Menu action	Shortcut	Purpose
Choose Scan Location...	Ctrl+O	Choose the scan source.
Browse Output File	Ctrl+Shift+O	Choose/view output destination.
Start Extraction	Ctrl+Enter	Start the extraction operation.
Cancel Extraction	Esc	Request cooperative cancellation.
Clear Log	—	Clear extraction log text.

Menu action	Shortcut	Purpose
Reset	—	Reset the interface state.
Exit	Ctrl+Q	Close Skim.
About Skim	—	Show application information.
About Qt	—	Show Qt information.

18. Branding and Product Identity

The application was renamed from DocumentExtractor to Skim at the product level. The executable target is named Skim, the application title/about presentation uses Skim, and the application settings namespace is being aligned with the new product name.

The project also contains a Windows icon resource and a Qt resource containing the application PNG icon. The runtime window icon is loaded through the Qt resource system.

Internal implementation names such as DocumentExtractorWorker can remain unchanged until there is a concrete maintenance reason to rename them. Product-facing consistency is more important than a large mechanical refactor.

19. Build System and Development Environment

Component	Current environment
Language	C++17
GUI framework	Qt 6.11.2
Build system	CMake 4.4.x
Compiler	MSYS2 MinGW GCC 16.2.0
Generator	MinGW Makefiles
PDF module	QtPdf / Qt6::Pdf
Word automation	Qt ActiveQt / Qt6::AxContainer
IDE/editor	Visual Studio Code

The CMake project requires Widgets, AxContainer, and Pdf. The current working configuration uses the Qt installation at D:/Dev/Tools/Qt/6.11.2/mingw_64 and the MinGW toolchain at C:/msys64/mingw64/bin.

```
cmake -S . -B build -G "MinGW Makefiles" ^
-DMAKE_C_COMPILER="C:/msys64/mingw64/bin/gcc.exe" ^
-DMAKE_CXX_COMPILER="C:/msys64/mingw64/bin/g++.exe" ^
-DMAKE_PREFIX_PATH="D:/Dev/Tools/Qt/6.11.2/mingw_64" cmake --build build
```

On PowerShell, the Qt runtime DLL directory may need to be available on PATH when launching the executable directly from the build directory.

20. Project Structure

```
documentExtractor/ ■■■ CMakeLists.txt ■■■ app_icon.rc ■■■ Resources/ ■■■  
DocumentExtractor_256.png ■■■ DocumentExtractor.ico ■■■ checkmark.svg ■■■  
DocumentExtractor.qss ■■■ src/ ■■■ main.cpp ■■■ MainWindow.cpp ■■■ MainWindow.h ■■■  
DocumentExtractor.cpp ■■■ DocumentExtractor.h
```

The exact directory layout can evolve, but the conceptual separation should remain: application entry point, GUI, extraction worker, and resources/build metadata.

21. Testing and Validation

The implementation has been exercised against the main workflows required for the project. The validation set included:

- 1 TXT-only extraction.
- 2 PDF-only extraction.
- 3 Word extraction.
- 4 Mixed extraction across supported formats.
- 5 Subfolder behavior.
- 6 Case-insensitive search.
- 7 Cancellation.
- 8 Output file located inside the input folder.
- 9 Bad/corrupt document handling.
- 10 Multiple-file selection.
- 11 Multiple drag-and-drop operations.
- 12 Files originating from different folders/drives.
- 13 Duplicate prevention.

The test philosophy is to verify both positive behavior (matches are found) and operational behavior (empty, invalid, duplicate, cancelled, and mixed-input cases are handled without losing the rest of the run).

22. Current Roadmap

The roadmap is intentionally organized around product maturity rather than adding features simply for the sake of adding them.

Stage	Status	Focus
Core extractor	Complete	TXT, Word, PDF and matching logic.
Reliability	Complete	Failures, cancellation, empty files, summaries.
GUI	Complete	Cards, QSS, progress, logs, dialogs.
Input expansion	Complete	Folder + multi-file workflows.
Branding	Mostly complete	Skim identity, icon, executable target.

Stage	Status	Focus
Final polish	Next	Cleanup, consistency, edge cases, release UX.
Packaging	Next	Create a repeatable distributable Windows build.
Documentation	This document	Technical and user-facing reference.

23. Release and Packaging Plan

Before distributing Skim, the project should move through a release checklist.

- Perform a clean configure/build from a fresh build directory.
- Run the complete functional test matrix again using the release build.
- Verify the Windows executable icon and application name.
- Verify Qt runtime deployment and required DLLs/plugins.
- Verify ActiveQt/Word behavior on the intended target machines.
- Verify PDF extraction with ordinary text PDFs and intentionally invalid PDFs.
- Verify output behavior when the output file already exists or is located under the scan directory.
- Verify cancellation while processing both ordinary files and Word/PDF inputs.
- Verify the application does not leave Word running after normal completion or cancellation.
- Package the executable with all required runtime components.
- Create a versioned release artifact and document the supported environment.

24. Future Enhancements

These are possible future improvements rather than requirements for the current stable roadmap.

- **OCR for image-only PDFs:** allow scanned documents without embedded text to participate in searches.
- **Richer search modes:** optional whole-word, regular-expression, or phrase matching if a concrete need emerges.
- **More output formats:** CSV/JSON or a richer report when downstream workflows justify them.
- **Configuration persistence:** remember useful non-sensitive UI preferences and recent locations.
- **Improved release packaging:** installer or self-contained deployment workflow.
- **Additional document formats:** only when they solve a real workflow requirement.
- **Accessibility improvements:** keyboard navigation, clearer focus states, and scalable presentation.
- **Performance optimization:** only after profiling real workloads.

25. Known Deliberate Non-Features

The clickable-filename experiment in the extraction log was intentionally abandoned. The idea was to turn filenames in the log into links that would open the source file through Windows. The implementation introduced unnecessary QTextEdit/QTextBrowser, anchor-signal, HTML, and MOC complexity relative to its value.

The stable design therefore keeps the extraction log as plain text. This is an example of an important engineering principle for Skim: an optional convenience feature should not be allowed to destabilize the core extraction pipeline or build system.

26. Developer Notes and Maintenance Guidance

The most important maintenance rule is to preserve the stable separation between UI and extraction.

Format-specific code should remain isolated, while the worker owns the orchestration and the main window owns presentation and user interaction.

- Do not perform long-running document extraction directly in the GUI thread.
- Keep cancellation cooperative and make cleanup deterministic.
- When changing Word automation, preserve the known working Documents.Open signature unless the target API is deliberately changed and tested.
- When changing PDF extraction, preserve page-aware reporting and explicit load-error handling.
- Do not let a single failed file terminate an otherwise valid batch.
- When adding GUI features, first consider whether the current widget class actually supports the required behavior.
- After substantial build-system changes, perform a clean configure/build before diagnosing source-level compiler errors.
- Keep the product name Skim consistent in user-facing surfaces while avoiding unnecessary internal renames.
- Maintain the existing test matrix whenever extraction behavior changes.

Appendix A — Stable Feature Checklist

- TXT, DOC, DOCX, PDF
- Case-insensitive matching
- Folder mode
- Files mode
- Recursive subfolders
- Multiple selected files
- Additive drag-and-drop
- Duplicate prevention
- Background worker
- Cancellation
- Progress

- Live log
- Output summary
- Failed-file list
- Empty-file handling
- Corrupt-document handling
- Qt stylesheet
- Application icon
- Windows executable icon
- Menus and shortcuts
- Completion dialog
- Reset and Clear Log

Appendix B — Product Direction

Skim has reached the point where the central extraction problem is solved and the application has a coherent desktop workflow. The remaining work should therefore emphasize stability, consistency, packaging, and documentation. New features should be added when they clearly improve the document-extraction workflow and can be implemented without compromising the reliability of the existing system.

End of Documentation